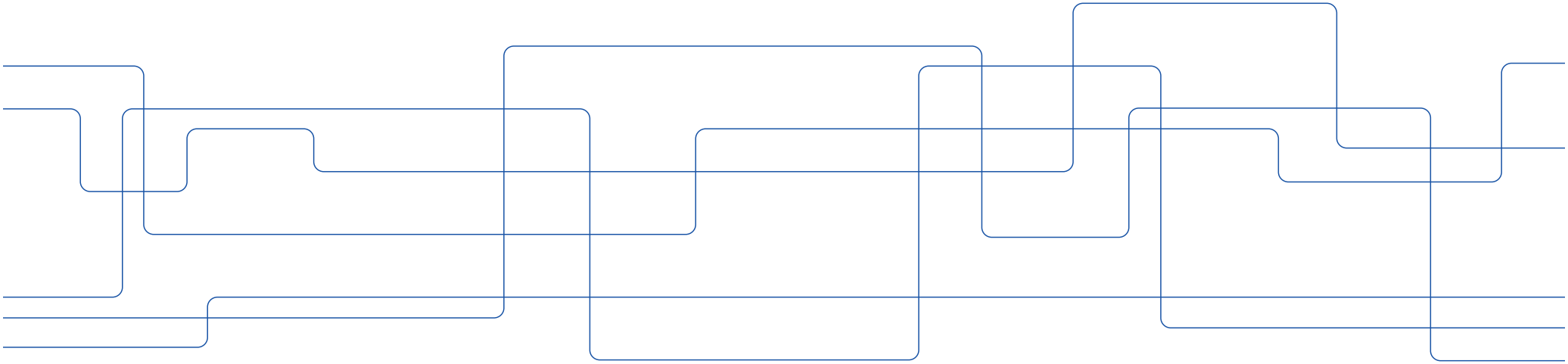



HI1024

# Grundkurs i programmering

## F7: Funktioner - introduktion



# Innehåll!

King 9.1..5

Repetition föreläsning #6

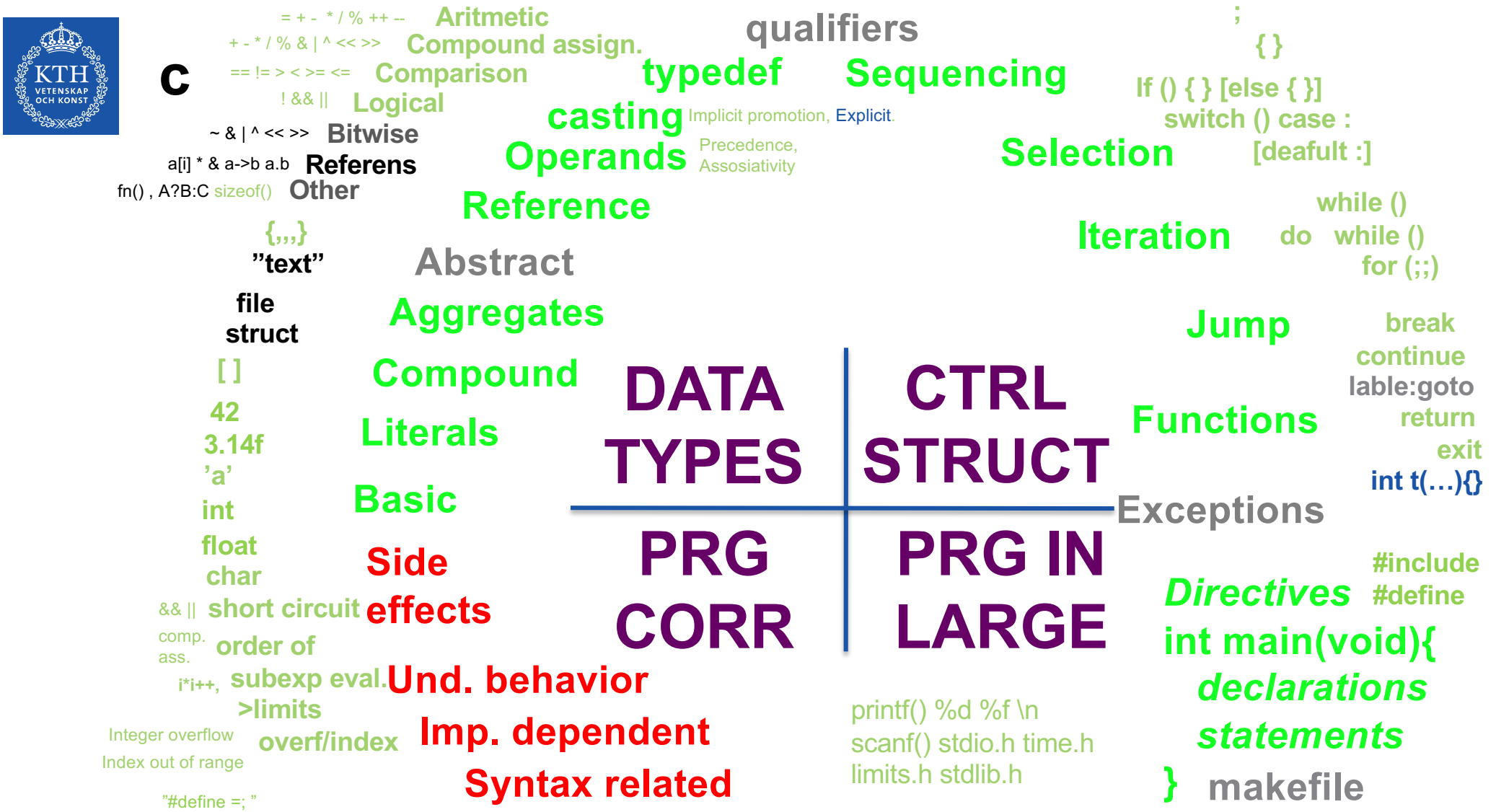
Funktioner - Introduktion

Programutvecklingsmodellen

Matriser som funktionsparametrar

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence,
Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
int/int
/*comment*/ or //

# Matriser…

Variabel box är en **matris** med WEEKDAY st element!

Alla element har samma typ!

```
int box[WEEKDAYS]={2,1,1,1,0,0,1};
```

Eventuell initiering! (om för få, fyller på 0:or)

Ett enskilt element nås via t.ex box[3], 3:an kallas index…

…och är då som en ”vanlig” variabel av elementtypen…

…det första elementet har indexet 0!

**Matriser måste kopieras/jämföras element för element!**

```
PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    2.     box[0]      int
#F001:    1.     box[1]      int
#F002:    1.     box[2]      Int
#F003:    1.     box[3]      int
#F004:    0.     box[4]      int
#F005:    0.     box[5]      int
#F006:    1.     box[6]      int
```

R






**…indexet kan vara vilket ”skumt” uttryck som helst bara resultatet blir ett positivt heltal!**

# Matriser…




Behöver du 1 int skriv: int number = 0;

Behöver du 7 int skriv: int week[7] = {0,1,2,3,4,5,6};



Behöver du 1 float skriv: float pi = 3.1415f;

Behöver du 3 float skriv: float test[3] = {2.2f, 2.3f, 2.1f};

Behöver du 1 char skriv: char unit = 'C';

Behöver du 4 char skriv: char name[4] = {'A','n','n','a'};

```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE     NAMN          TYP
#F000:   'A'      name[0]       char
#F001:   'n'      name[1]       char
#F002:   'n'      name[2]       char
#F003:   'a'      name[3]       char
#F004:
#F005:
#F006:
```

R






Kom ihåg: Första elementet har index 0, för att nå ett element använd [ ], t.ex. name[0] som är 'A'

# Övning: Palindrom

```
// WORDMAX 20

// word[WORDMAX]
// sista -1

// Programrubrik

// Instruktion ange ord
// Repetera…
//   läs in bokstav till ord*
// ...tills return

// Visa rubrik
// För alla tecken i ord
//    skriv ut bakifrån
// Gör ny rad

// Klart!

*öka sista med 1 i smyg!
```

```c
/* Characters 0.3 AC */
#include <stdio.h>
#define WORDMAX 20

int main(void){
    char word[WORDMAX];
    int last=-1;

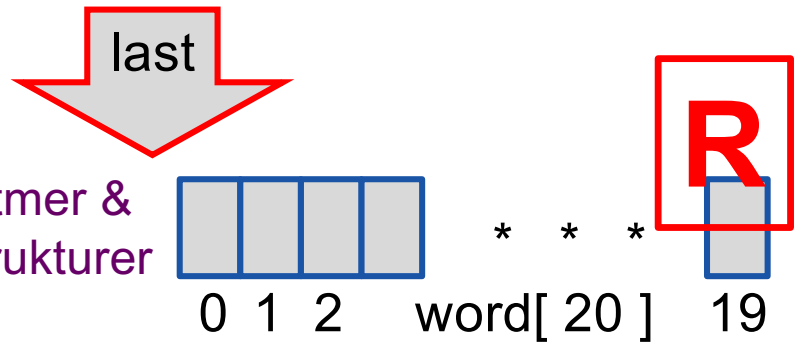
    printf("Matrix Tester (MT)!\n");

    printf("Enter a word   :");
    do
       scanf("%c", &word[++last]);
    while (word[last]!='\n');

    printf("it is reversed :");
    for (int i=last-1; i>=0; i--)
        printf("%c",word[i]);
    printf("\n");

    printf("MT Done.\n");
    return 0;}
```

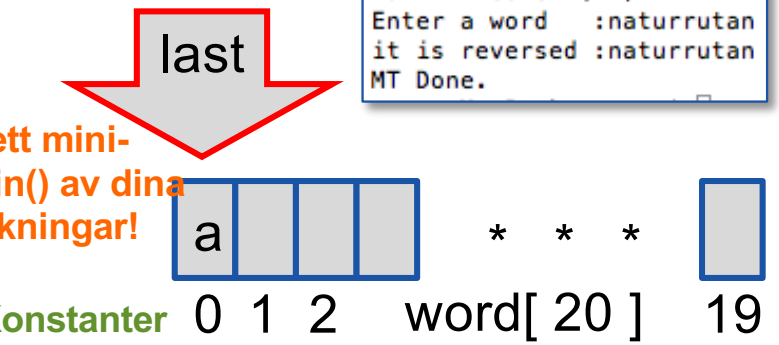
**0** Algoritmer & Datastrukturer




**1** Skriv ner problemet så att ett barn förstår det!

```
Matrix Tester (MT)!
Enter a word   :naturrutan
it is reversed :naturrutan
MT Done.
```

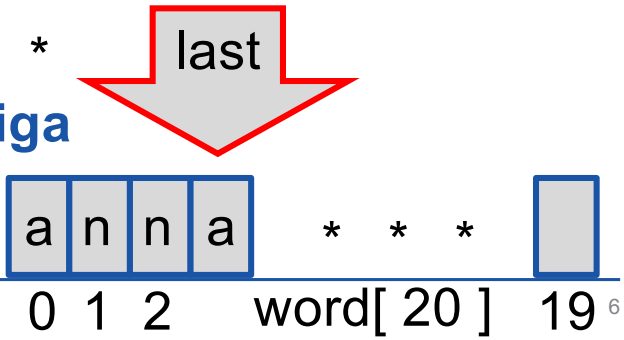
**2** Gör ett mini-malt main() av dina anteckningar!




**3** Konstanter
Variabler
Uttryck &
resultat!
a..n




**4** Läs in riktiga värden!



# Den här då?




”rader”

```
int w[4][7]={{1,2,3,4,5,6,7},
```

”kolumner”

```
             {1,2,3,4,5,6,7},
             {1,2,3,4,5,6,7},
             {1,2,3,4,5,6,7}};
```

**Wow!**

R

```
PROGRAM
#0000: CODE
   *
DATA   VÄRDE    NAMN        TYP
#F000:    1     w[0][0]     int
#F001:    2     w[0][1]     int
#F002:    3     w[0][2]     int
#F003:    4     w[0][3]     int
#F004:    5     w[0][4]     int
#F005:    6     w[0][5]     int
#F006:    7     w[0][6]     int
#F007:    1     w[1][0]     int
#F008:    2     w[1][1]     int
#F009:    3     w[1][2]     int
#F00A:    4     w[1][3]     int
#F00B:    5     w[1][4]     int
#F00C:    6     w[1][5]     int
#F00D:    7     w[1][6]     int
#F00E:    1     w[2][0]     int
```

Rad 0

Rad 1

Det går att ha fler dimensioner också:
**char book[PAGES][ROWS][COLUMNS];**



**Observera hur klamrarna sätts, det ska vara w[rad][kolumn] (inte [rad, kolumn])**

# Nytt: Slumptal!



Det finns ingen dator som är bra på slump…

…men det går ofta på något sätt att efterlikna slump…

…i c-biblioteket **stdlib.h** finns funktionen **srand()** som med hjälp av ett slumpfrö initierar en slumptalsgenerator. Slumptalen, som är mellan 0 och maxint, hämtas med funktionen **rand()**. Slumpfröet brukar ofta vara datorns klocka och tiden hämtas via **time.h** funktionen **time()**. Satserna blir: **srand( (unsigned)time(NULL) ); och rnd = rand();**

(Det går att köra utan att srand() anropats, men då blir slumpserien exakt lika varje gång programmet körs!)

# Nytt: Slumptal!



```c
/* Characters 0.6 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    srand((unsigned) time(NULL));    /* Init random generator */

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

```
Matrix Tester (MT)!
Dice 1 : 6
Dice 2 : 2
Dice 3 : 1
Dice 4 : 2
Dice 5 : 4
MT Done.
vcxxs-macbook:~ vcxx
Matrix Tester (MT)!
Dice 1 : 2
Dice 2 : 3
Dice 3 : 5
Dice 4 : 2
Dice 5 : 2
MT Done.
vcxxs-macbook:~ vcxx
Matrix Tester (MT)!
Dice 1 : 4
Dice 2 : 3
Dice 3 : 1
Dice 4 : 1
Dice 5 : 2
MT Done.
```

**Yatzy 0.1**

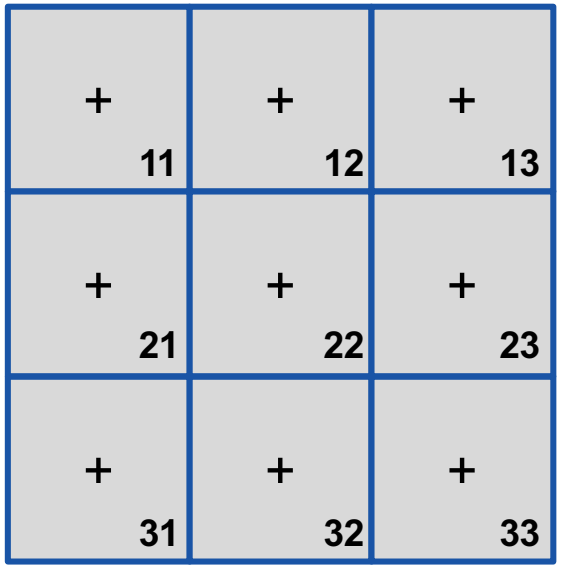


**rand() ger ett tal mellan 0 och maxint, men ta modulus (%) 6 så får du ett tal mellan 0 och 5, perfekt!**

# Reflektioner övning #6

**0**

Algoritmer & Datastrukturer




Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Reflektioner övning #6

**1** **Skriv ner problemet så att ett barn förstår det!**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

Repetera…
  Visa spelplanen!

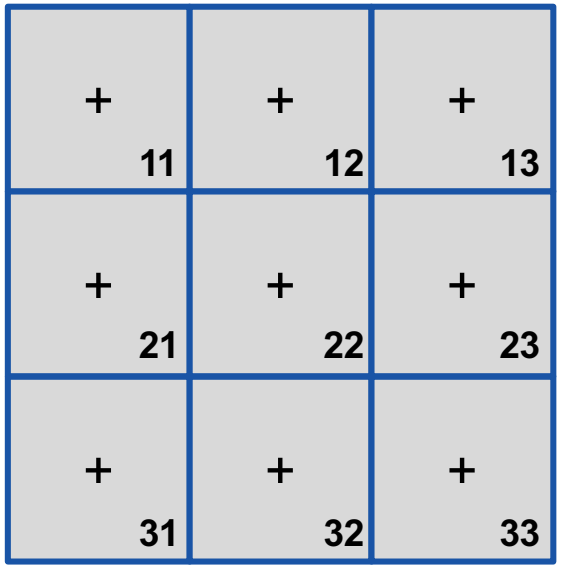
  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
   Om turn++ udda
      row,col X else
      row,col O

…så länge turn>0

**0**

Algoritmer & Datastrukturer




Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Reflektioner övning #6

**1** **Skriv ner problem så att ett barn förstår det!**

**2**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

Repetera…
  Visa spelplanen!

  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
   Om turn++ udda
      row,col X else
      row,col O

…så länge turn>0

```c
#include <stdio.h>

int main(int argc, char **argv){


    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");


    printf("Done, bye!\n");
    return 0;
}
```

**Gör ett mini-malt main() av dina anteckningar!**

**0**

Algoritmer & Datastrukturer

| | | |
|---|---|---|
| + 11 | + 12 | + 13 |
| + 21 | + 22 | + 23 |
| + 31 | + 32 | + 33 |

Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Reflektioner övning #6

**1** **Skriv ner problem så att ett barn förstår det!**

**2**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

Repetera…
  Visa spelplanen!

  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
   Om turn++ udda
      row,col X else
      row,col O

…så länge turn>0

**3**

**Konstanter**
**Variabler**
**Uttryck & resultat!**

1..3
n

```c
#include <stdio.h>
#define S 3

int main(int argc, char **argv){
    char b[S][S]={{'+','+','+'},
                  {'+','+','+'},
                  {'+','+','+'}};
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");

    printf("Done, bye!\n");
    return 0;
}
```

**0**

Algoritmer & Datastrukturer

| | | |
|---|---|---|
| + 11 | + 12 | + 13 |
| + 21 | + 22 | + 23 |
| + 31 | + 32 | + 33 |

Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Reflektioner övning #6

**0**

Algoritmer & Datastrukturer

**1** **Skriv ner problem så att ett barn förstår det!**

**2**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

**3**

Repetera…
  Visa spelplanen!

  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
    Om turn++ udda
       row,col X else
       row,col O

…så länge turn>0

```
#incl#incl#include <stdio.h>
     #defi#define S 3

int mint mint main(int argc, char **argv){
          c     char b[S][S]={{'+','+','+'},
                              {'+','+','+'},
                              {'+','+','+'}};
          c     char r='0',c='0'; int turn=0;

     p    p     printf("Tic Tac Toe!\n");
     p    p     printf("Enter rowcol\n");
     p    p     printf("or 00 to end\n");

               for (int i=0; i<S; i++) {
                  for (int j=0; j<S; j++)
                     printf("%c", b[i][j]);
                  printf("\n");
               }

     p    p     printf("Done, bye!\n");
     r    r     return 0;
}    }    }
```

1..3 r Konstanter
m 2..3 Variabler
Uttryck & resultat!

| | | |
|---|---|---|
| + 11 | + 12 | + 13 |
| + 21 | + 22 | + 23 |
| + 31 | + 32 | + 33 |

Hum: char b[3][3]
Initierat med ’+’?!

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```



Så fort du har en ny datastruktur, se till att du kan skriva ut den!

**(Men i c [0..2][0..2]!)**

# Reflektioner övning #6

**1** **Skriv ner problem så att ett barn förstår det!**

**2**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

**3**

Repetera…
  Visa spelplanen!

  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
   Om turn++ udda
      row,col X else
      row,col O

…så länge turn>0

```
#include <stdio.h>
#define S 3

int main(int argc, char **argv){
    char b[S][S]={{'+','+','+'},
                  {'+','+','+'},
                  {'+','+','+'}};
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    do {
       for (int i=0; i<S; i++) {
          for (int j=0; j<S; j++)
             printf("%c", b[i][j]);
          printf("\n");
       }

1..3
2..3
3..3   Uttryck

    if (r<'1' || c<'1')
        turn=0;
    else {
        if (++turn%2)
            b[r-'1'][c-'1']='X';
        else
            b[r-'1'][c-'1']='0';
    }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```

**0**

Algoritmer &
Datastrukturer

| | | |
|---|---|---|
| + 11 | + 12 | + 13 |
| + 21 | + 22 | + 23 |
| + 31 | + 32 | + 33 |

Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Reflektioner övning #6

**0** Algoritmer & Datastrukturer

**1** Skriv ner problem så att ett barn förstår det!

**2**

b[SIZE][SIZE] +
row,col,turn=0

Välkommen!

**3**

Repetera…
  Visa spelplanen!

  Vart row,col?

 Om rowcol är 00
    turn=0
 i annat fall
    Om turn++ udda
       row,col X else
       row,col O

…så länge turn>0

```c
#include <stdio.h>
#define S 3

int main(int argc, char **argv){
    char b[S][S]={{'+','+','+'},
                  {'+','+','+'},
                  {'+','+','+'}};
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    do {
       for (int i=0; i<S; i++) {
          for (int j=0; j<S; j++)
             printf("%c", b[i][j]);
          printf("\n");
       }

       printf("Where?");
       scanf(" %c%c",&r,&c);

       if (r<'1' || c<'1')
          turn=0;
       else {
          if (++turn%2
             b[r-'1'][c-'1']='X';
          else
             b[r-'1'][c-'1']='O';
       }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```

**4** Läs in riktiga värden!

| | | |
|---|---|---|
| + 11 | + 12 | + 13 |
| + 21 | + 22 | + 23 |
| + 31 | + 32 | + 33 |

Hum: char b[3][3]
Initierat med ’+’?!

**(Men i c [0..2][0..2]!)**

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

# Nytt: Funktioner

Start

**Köttbullar**

Visst finns det goda köttbullar i affärernas frysdiskar, men ingenting går upp mot de hemlagade! Variera med olika smaksättningar.

TILL 4 PERSONER:

1 sats grundrecept på färs (se s.339)
2 msk smör

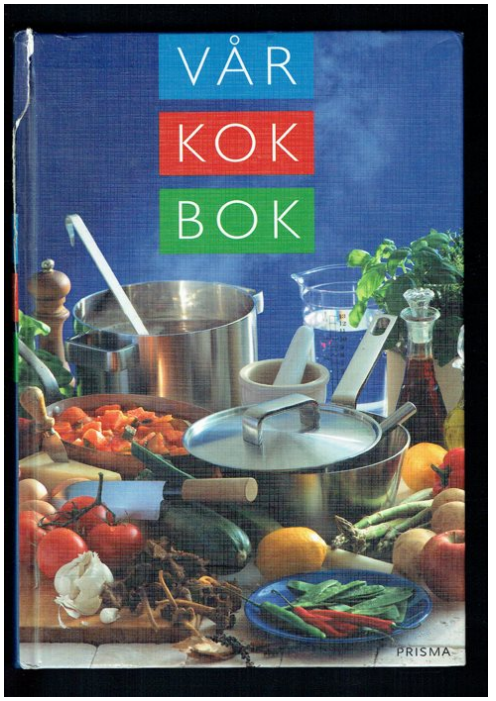
Klart

Den här texten hade lika gärna kunnat stå där referensen står i texten till vänster…

TILL 4 PERSONER:

400 g blandfärs eller annan färs
4 msk strö bröd
1 1/2 dl mjölk eller vatten
1 ägg
1 1/2 tsk salt, 1 krm vitpeppar
1–2 msk råriven eller finhackad lök (ev. fräst)

…men av olika skäl har texten fått ett eget namn (grundrecept) och flyttats till en annan plats!






**Den här uppdelningen i sig, gör instruktionen mer lättläst…**

# Nytt: Funktioner

Start

## Köttbullar

Visst finns det goda köttbullar i affärernas frysdiskar, men ingenting går upp mot de hemlagade! Variera med olika smaksättningar.

TILL 4 PERSONER:

1 sats grundrecept på färs (se s.339)
2 msk smör

Klart

Vilket innebär att andra recept också kan ref. till samma text, utan att den behöver upprepas!

TILL 4 PERSONER:

400 g blandfärs eller annan färs
4 msk ströbröd
1 1/2 dl mjölk eller vatten
1 ägg
1 1/2 tsk salt, 1 krm vitpeppar
1–2 msk råriven eller finhackad lök (ev. fräst)

**Wow, återanvända instruktioner!**

## Köttfärslimpa

Ett lätt och smidigt sätt att laga till köttfärs. Passa på att steka potatis och rotsaker i ugnen (se s. 156) samtidigt med köttfärslimpan.

Smaksätt gärna färsen med olika kryddor. Se till exempel recepten på köttbullar på sidan 339.

TILL 4 PERSONER:

1 sats grundrecept på färs (se s. 339)
2 msk smält smör

Sätt ugnen på 175°.

Blanda färsen enligt grundrecept men öka mängden mjölk eller vatten till 2 dl. Forma färsen till en limpa på ett smort, ugnssäkert fat.

Stek mitt i ugnen i 45–60 minuter. Pensla med smöret ett par gånger under stekningen.

Klart

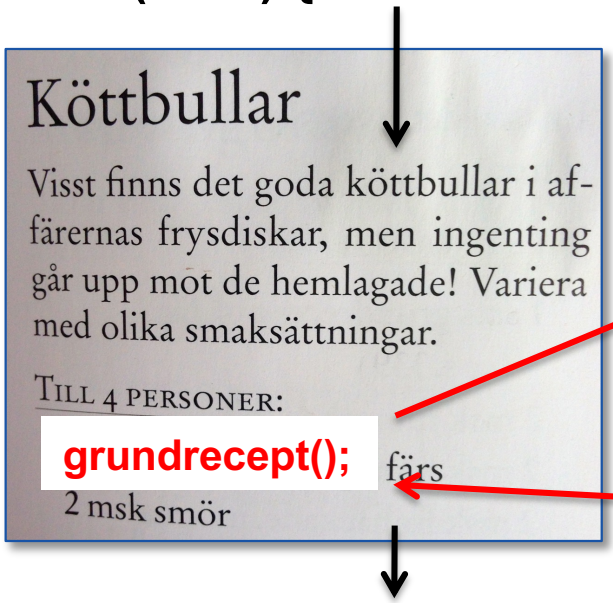


**…att den sedan också gör att instruktionen kan delas är otroligt bra!**

# Nytt: Funktioner

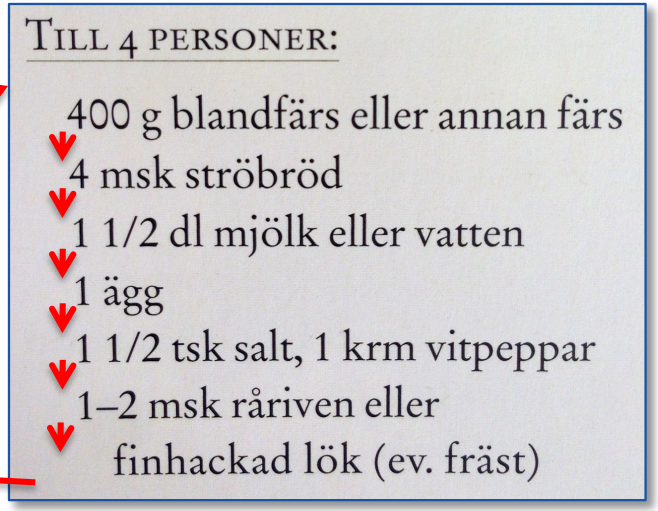
**Att skapa en funktion i c är enkelt: Inneslut önskade satser i { } och skriv void namn(void) före, så kan raderna anropas från main med namn();**

```
int main(void) {
```




```
}
```

```
void grundrecept(void) {
```




```
}
```



**Hur vet vi om några satser borde vara en egen funktion?**

# Nytt: Funktioner

**int** main(**void**) = Huvudfunktionen!

```c
/* Characters 0.6 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    srand((unsigned) time(NULL));   /* Init random generator */

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

När du skriver kodsegment som:

- Är svår att överblicka/krångliga…
- Återkommer…
- Återanvändbar…

Så är det funktionskandidater!

**Finns det dylik kod i exemplet?**

# Nytt: Funktioner

```c
/* Characters 0.6 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    srand((unsigned) time(NULL));   /* Init random generator */

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

Obegripligt!

Återanvändbart!

Återkommande!

Återanvändbart!



**Om vi ”bryter ut” de 2 kodpartierna, behöver vi tänka på något mer?**

# Nytt: Funktioner

```c
/* Characters 0.6 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    srand((unsigned) time(NULL));   /* Init random generator */

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

**Om jag ”bryter ut” satserna till en egen funktion…**
**…behöver jag då skicka med något (in-data)?**
**…eller måste jag få något tillbaks (ut-data)?**

Obegripligt!
Återanvändbart!

**Ingen in-data!**
**Ingen ut-data!**

Återkommande!
Återanvändbart!

**Ingen in-data!**
**Ut-data: kast!**



**Vi provar med srand()-satsen…**

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

Måste (i detta fall) stå före (första) anropet!

**Ingen ut-data!** **Namn** **Ingen in-data!**

```c
void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}
```

Benämns: Funktionsdefinition

Lättbegripligt!

~~Obegripligt!~~

Återanvändbart.

**Ingen in-data!**

**Ingen ut-data!**

# Nytt: Funktioner

Ett typiskt c-program innehåller 100-tals, för att inte säga 1000-tals funktioner…

…för att inte main() ska hamna sist så är **Best Practice** att berätta för kompilatorn att funktionen finns genom en så kallad funktionsdeklaration som är funktionsdefinitionen fram till ’{’ som ersätts med ett ’;’…

…informationen är nog för att kompilatorn ska kunna göra sitt arbete, i förvissningen om att en fullständig funktionsdefinition kommer innan filen är slut!

**Funktionsdefinition** →

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}

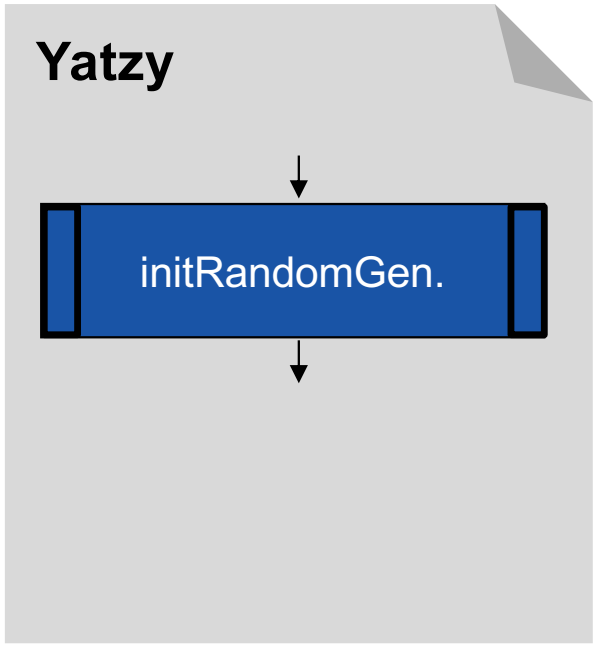
void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}
```

← **Funktionsdeklaration**

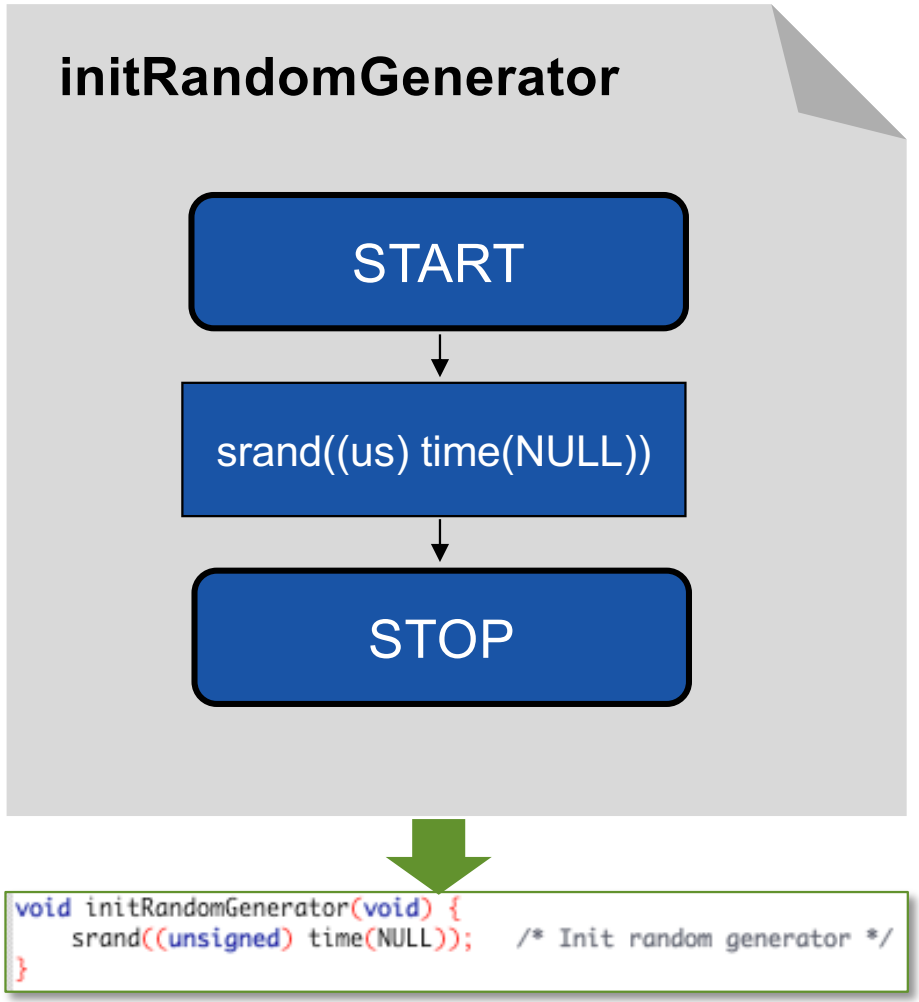


Vi vill att kod som ni redovisar ser ut på detta sätt. Under utveckling är det ok att börja med enbart definitionen.

# Nytt: Funktioner




(Flowchart)






Ok, kan vi även ”bryta ut” den misstänkt återanvändbara koden för ett tärningskast?

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}
```

Återkommande!
Återanvändbart!
**Ingen in-data!**
**Ut-data: kast!**

För att kunna ”bryta ut” den här koden måste den nya funktionen kunna ”sända tillbaks” ett heltal, en int…

…det kan göras på flera sätt i c men är det enbart en int, float eller char så kan själva funktionen returnera det värdet…

…om en funktion returnerar ett int-värde så byts datatypen före funktionsnamnet ut från void (=returnerar inget) till int (=returnerar ett heltal). I funktionen måste det nu finnas minst en return-sats. När den utförs sker ett återhopp till den anropande koden och det värde som står efter return returneras:

```c
int  throwDice(void) {
    return (rand() % SIZE) + 1;
}
```

# Nytt: Funktioner

Funktionsdeklaration

Funktionsanrop

Funktionsdefinition

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice());

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(void) {
    return (rand() % SIZE) + 1;
}
```

# Nytt: Funktioner

## Observationer!

Returtypen kan vara int, float, char men inte array!

```
int  throwDice(void) {
    int throw=(rand() % SIZE) + 1;

    return throw;
}
```

Precis som main kan throwDice ha egna variabler. De existerar dock bara när datorn kör funktionen throwDice! Och de döljer då andra eventuella variabler med samma namn*!

c fungerar enligt principen copy-by-value. Det betyder att när return ska returnera ett värde så beräknar den först värdet av throw, säg 5, som den därefter returnerar. Den returnerar m.a.o en ”kopia” av resultatet (och inte en referens till throw, för throw finns inte när funktionen är avslutad!)

(*Kommer aldrig att hända i denna kurs, gäller enbart globala variabler.)

# Nytt: Funktioner

throwDice() skulle kunna bli riktigt generell och återanvändbar om vi kunde ”sända med”, vid grön pil, hur många ”ögon” tärningen ska ha som ska kastas…

…det går att skriva ett tal där (t.ex 6), eller ett uttryck som när det beräknas blir 6, det talet eller uttrycket kallas för argument…

…vid den röda pilen så måste vi definiera en variabel av heltalstyp som kan ”ta emot” 6:an som fanns som argument i anropet, t.ex int size, som kallas parameter…

…en parameter fungerar som en lokal variabel i funktionen, men bara där!

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(void);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice());

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(void) {
    return (rand() % SIZE) + 1;
}
```

(int size) OBS: Ny variabel!

6 eller SIZE

(int size) OBS: Ny variabel!



Observera: Parametern får sitt värde via copy-by-value, så originalet kan inte påverkas!

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE     NAMN          TYP
#F000:    ?         ?            ?
#F001:    ?         ?            ?
#F002:    ?         ?            ?
#F003:
#F004:
#F005:
#F006:
```

Ok, här börjar visst programmet…

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE     NAMN          TYP
#F000:    ?        ?             ?
#F001:    ?        ?             ?
#F002:    ?        ?             ?
#F003:
#F004:
#F005:
#F006:
```

Skriv lite text…

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE     NAMN          TYP
#F000:    ?         ?            ?
#F001:    ?         ?            ?
#F002:    ?         ?            ?
#F003:
#F004:
#F005:
#F006:
```

**Kom ihåg det här stället…**

**…stick iväg och utför instruktionerna i funktionen initRandomGenerator…**

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

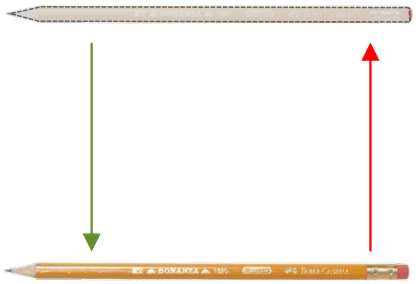
    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE     NAMN          TYP
#F000:    ?        ?             ?
#F001:    ?        ?             ?
#F002:    ?        ?             ?
#F003:
#F004:
#F005:
#F006:
```

…ok, slumpgeneratorn initierad…

…jag fortsätter där jag var!

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0        i         int
#F001:    ?        ?          ?
#F002:    ?        ?          ?
#F003:
#F004:
#F005:
#F006:
```

…behöver en ny variabel i som är 0…

…den är < HAND så fortsätt…

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0        i          int
#F001:    ?        ?           ?
#F002:    ?        ?           ?
#F003:
#F004:
#F005:
#F006:
```

…jobbigt, jag ska skriva ut saker…

…men först måste jag anropa
throwDice och sända med en 6:a…

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0        i         int
#F001:    6      size        int
#F002:    ?        ?          ?
#F003:
#F004:
#F005:
#F006:
```

…ny variabel size som ska vara 6…



**Observera att size är en ny (lokal) (parameter-)variabel, som får en egen plats i minnet!**

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0       i          int
#F001:    6      size        int
#F002:    ?       ?           ?
#F003:
#F004:
#F005:
#F006:
```

…beräkna uttrycket, blir t.ex. 3, sänd tillbaks svaret och glöm allt!



Obs: Det går att ändra värdet på size i throwDice, men det påverkar inte argumentet (tur det då det är en konstant)!

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```



```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0        i          int
#F001:    ?        ?           ?
#F002:    ?        ?           ?
#F003:
#F004:
#F005:
#F006:
```

…till slut, skriv ut den returnerade 3:an, det var på tiden!

# Nytt: Funktioner

```c
/* Functions 0.1 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

void initRandomGenerator(void);
int  throwDice(int size);

int main(void){

    printf("Matrix Tester (MT)!\n");

    initRandomGenerator();

    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, throwDice(SIZE));

    printf("MT Done.\n");
    return 0;
}

void initRandomGenerator(void) {
    srand((unsigned) time(NULL));   /* Init random generator */
}

int  throwDice(int size) {

    return (rand() % size) + 1;
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0 1      i          int
#F001:    ?        ?           ?
#F002:    ?        ?           ?
#F003:
#F004:
#F005:
#F006:
```

…öka i med ett, är jag klar?

…nej gör om allt en gång till…

…osv

**Sensmoral: Döp aldrig parametrar till samma namn som anropande argument, det är olika variabler!**




# Nytt: Funktioner

Med andra ord: En funktion ska om möjligt vara ett fåtal antal satser som utför någon ofta återkommande, specifik, uppgift! Behovet av en funktion identifieras lättas:

- Under arbetet med en funktion (i vårt fall main()) noterar vi att det finns:
    1. Komplicerad/Obegriplig kod
    2. Återanvändbar kod
    3. Funktionen håller på att bli för stor (mer än en skärmsida).

- Under ”skissarbetet”, innan vi skriver någon kod, inser vi att det finns en handfull funktioner som skulle göra allt mycket enklare – ofta kopplat till de datastrukturer programmet kommer att använda sig av!




# Nytt: Funktioner

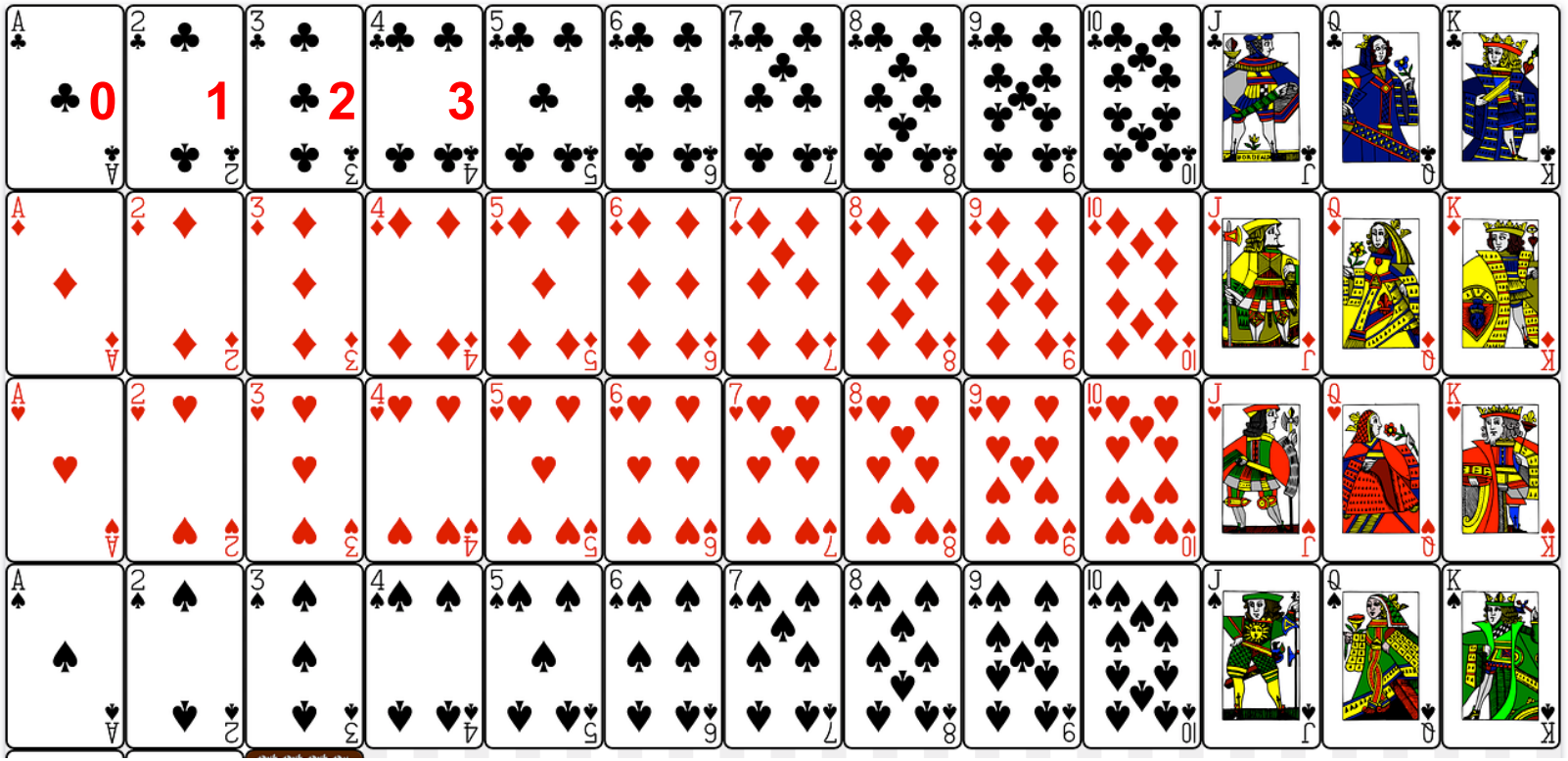
En funktion utvecklas på samma sätt som när vi har skapat huvudfunktionen main()!

- Behöver funktionen någon speciell datastruktur/algoritm (0)
- Skissa en lösning till funktionens kod! (1)
- Skapa enbart anropet, och testa att det fungerar! (2)
- Utveckla funktionen, steg för steg, testa! (3..4)
- (Spara testprogrammet som testar funktionen!)
- Använd funktionen!



**Observera: Den första koden som skrivs är testprogrammet som kan anropa den blivande funktionen!**

# Övning: Skriv ut en kortlek II




```
Monte Carlo 0.3
[Club    A]
[Club    2]
[Club    3]
[Club    4]
[Club    5]
[Club    6]
[Club    7]
[Club    8]
[Club    9]
[Club    T]
[Club    J]
[Club    Q]
[Club    K]
[Diamond A]
[Diamond 2]
[Diamond 3]
[Diamond 4]
[Diamond 5]
[Diamond 6]
[Diamond 7]
[Diamond 8]
[Diamond 9]
[Diamond T]
[Diamond J]
[Diamond Q]
[Diamond K]
[Heart   A]
[Heart   2]
[Heart   3]
[Heart   4]
[Heart   5]
[Heart   6]
[Heart   7]
[Heart   8]
[Heart   9]
[Heart   T]
[Heart   J]
[Heart   Q]
[Heart   K]
[Spade   A]
[Spade   2]
[Spade   3]
[Spade   4]
[Spade   5]
[Spade   6]
[Spade   7]
[Spade   8]
[Spade   9]
[Spade   T]
[Spade   J]
[Spade   Q]
[Spade   K]
```



Skapa en rutin printCard() som skriver ut färg och valör… ...vi tänker oss vidare att korten är numrerade 0..51!

# Övning: Skriv ut en kortlek II

**0** Algoritmer & Datastrukturer

...det borde inte vara svårare än att färgen är kortnumret / 4 och valören kortnumret % 13

**1** **Skriv ner problemet så att ett barn förstår det!**

// KF 13 (Kort i en färg)

// Färg nr/KF
// Valör nr%KF

**2** **Gör ett mini-malt main() av dina anteckningar!**

**M.a.o fixa först ett testprogram!**

```c
#include <stdio.h>

// Deck of Card toolbox interface ------
#define SUITS  4
#define RANK  13
#define DECK  52

int main(int argc, char **argv)
{
    printf("Monte Carlo 0.1\n");

    for (int card=0; card<DECK; card++)
        printf("[%d]\n",card);

    printf("Program completed, bye!\n");
    return 0;
}
```

```
Monte Carlo 0.1
[0]
[1]
[2]
[3]
[4]
[5]
[6]
[7]
[8]
[9]
[10]
[11]
[12]
[13]
[14]
[15]
[16]
[17]
[18]
[19]
[20]
[21]
[22]
[23]
[24]
[25]
[26]
[27]
[28]
[29]
[30]
[31]
[32]
[33]
[34]
[35]
[36]
[37]
[38]
[39]
[40]
[41]
[42]
[43]
[44]
[45]
[46]
[47]
[48]
[49]
[50]
[51]
```

**OK!**



**Ok, det här ser bra ut… …hur ser funktionsdeklarationen ut för printCard()?**

# Övning: Skriv ut en kortlek II

**0** Algoritmer & Datastrukturer

...det borde inte vara svårare än att färgen är kortnumret / 4 och valören kortnumret % 13

**1** **Skriv ner problemet så att ett barn förstår det!**

// KF 13 (Kort i en färg)

// Färg nr/KF
// Valör nr%KF

**2** **Gör ett mini-malt main() av dina**

**M.a.o fixa först ett**

```c
#include <stdio.h>

// Deck of Card toolbox interface ------
#define SUITS  4
#define RANK  13
#define DECK  52
void printCard(int aCard){
    printf("[%d]\n",aCard);
}

int main(int argc, char **argv)
{
    printf("Monte Carlo 0.2\n");

    for (int card=0; card<DECK; card++)
        printCard(card);

    printf("Program completed, bye!\n");
    return 0;
}
```

```
Monte Carlo 0.2
[0]
[1]
[2]
[3]
[4]
[5]
[6]
[7]
[8]
[9]
[10]
[11]
[12]
[13]
[14]
[15]
[16]
[17]
[18]
[19]
[20]
[21]
[22]
[23]
[24]
[25]
[26]
[27]
[28]
[29]
[30]
[31]
[32]
[33]
[34]
[35]
[36]
[37]
[38]
[39]
[40]
[41]
[42]
[43]
[44]
[45]
[46]
[47]
[48]
[49]
[50]
[51]
```

**OK!**

**Ok, det här ser fortfarande bra ut, parametern har rätt värde…**

# Övning: Skriv ut en kortlek II

**0** Algoritmer & Datastrukturer

...det borde inte vara svårare än att färgen är kortnumret / 4 och valören kortnumret % 13

**1** **Skriv ner problemet så att ett barn förstår det!**

// KF 13 (Kort i en färg)

// Färg nr/KF
// Valör nr%KF

**2** **Gör ett mini-malt main() av dina** **M.a.o fixa först ett**

```c
#include <stdio.h>

// Deck of Card toolbox interface ------
#define SUITS  4
#define RANK  13
#define DECK  52
void printCard(int aCard){
    printf("[%d]\n",aCard);
}

int main(int argc, char **argv)
{
    printf("Monte Carlo 0.2\n");

    for (int card=0; card<DECK; card++)
        printCard(card);

    printf("Program completed, bye!\n");
    return 0;
}
```

**3** **Konstanter Variabler Uttryck & a..n resultat!**

```
Monte Carlo 0.2
[0]
[1]
[2]
[3]
[4]
[5]
[6]
[7]
[8]
[9]
[10]
[11]
[12]
[13]
[14]
[15]
[16]
[17]
[18]
[19]
[20]
[21]
[22]
[23]
[24]
[25]
[26]
[27]
[28]
[29]
[30]
[31]
[32]
[33]
[34]
[35]
[36]
[37]
[38]
[39]
[40]
[41]
[42]
[43]
[44]
[45]
[46]
[47]
[48]
[49]
[50]
[51]
```

**OK!**

**…dags att fixa till innehållet!**

# Övning: Skriv ut en kortlek II

**0** Algoritmer & Datastrukturer

...det borde inte vara svårare än att färgen är kortnumret / 4 och valören kortnumret % 13

**1** **Skriv ner problemet så att ett barn förstår det!**

// KF 13 (Kort i en färg)

// Färg nr/KF
// Valör nr%KF

**2**

**3** Konstanter
Variabler
Uttryck & resultat!

**4** Läs in riktiga värden!

```c
#include <stdio.h>

// Deck of Card toolbox interface ---------
#define SUITS  4
#define RANK  13
#define DECK  52
void printCard(int aCard){
    switch (aCard/RANK) {
        case 0: printf("[Club    "); break;
        case 1: printf("[Diamond "); break;
        case 2: printf("[Heart   "); break;
        case 3: printf("[Spade   "); break;
    }
    switch (aCard%RANK) {
        case  0: printf("A] "); break;
        case  9: printf("T] "); break;
        case 10: printf("J] "); break;
        case 11: printf("Q] "); break;
        case 12: printf("K] "); break;
        default: printf("%d] ",(aCard%RANK)+1); break;
    }
}
```

```
Monte [0]  Monte [0]  Monte Carlo 0.3
[0]   [0]   [Club    A]
[1]   [1]   [Club    2]
[2]   [2]   [Club    3]
[3]   [3]   [Club    4]
[4]   [4]   [Club    5]
[5]   [5]   [Club    6]
[6]   [6]   [Club    7]
[7]   [7]   [Club    8]
[8]   [8]   [Club    9]
[9]   [9]   [Club    T]
[10]  [10]  [Club    J]
[11]  [11]  [Club    Q]
[12]  [12]  [Club    K]
[13]  [13]  [Diamond A]
[14]  [14]  [Diamond 2]
[15]  [15]  [Diamond 3]
[16]  [16]  [Diamond 4]
[17]  [17]  [Diamond 5]
[18]  [18]  [Diamond 6]
[19]  [19]  [Diamond 7]
[20]  [20]  [Diamond 8]
[21]  [21]  [Diamond 9]
[22]  [22]  [Diamond T]
[23]  [23]  [Diamond J]
[24]  [24]  [Diamond Q]
[25]  [25]  [Diamond K]
[26]  [26]  [Heart   A]
[27]  [27]  [Heart   2]
[28]  [28]  [Heart   3]
[29]  [29]  [Heart   4]
[30]  [30]  [Heart   5]
[31]  [31]  [Heart   6]
[32]  [32]  [Heart   7]
[33]  [33]  [Heart   8]
[34]  [34]  [Heart   9]
[35]  [35]  [Heart   T]
[36]  [36]  [Heart   J]
[37]  [37]  [Heart   Q]
[38]  [38]  [Heart   K]
[39]  [39]  [Spade   A]
[40]  [40]  [Spade   2]
[41]  [41]  [Spade   3]
[42]  [42]  [Spade   4]
[43]  [43]  [Spade   5]
[44]  [44]  [Spade   6]
[45]  [45]  [Spade   7]
[46]  [46]  [Spade   8]
[47]  [47]  [Spade   9]
[48]  [48]  [Spade   T]
[49]  [49]  [Spade   J]
[50]  [50]  [Spade   Q]
[51]  [51]  [Spade   K]
```

Wow, ok!

# Nu behöver vi veta kortets värde?!

**0** Algoritmer & Datastrukturer **2**

**Gör ett mini-malt main() av dina anteckningar!**

**M.a.o fixa först ett testprogram!**

...det borde inte vara svårare än att returnera 10 för allt utom ess..9!

**1** **Skriv ner problemet så att ett barn förstår det!**

```
// RAM 10 (Kn,D & K)

// om nb>=RAM 10
// i annat fall nb+1
```



**Ok, det här ser bra ut… …hur ser funktionsdeklarationen ut för getRank()?**

# Nu behöver vi veta kortets värde?!

**0** Algoritmer & Datastrukturer

...det borde inte vara svårare än att returnera 10 för allt utom ess..9!

**1** **Skriv ner problemet så att ett barn förstår det!**

// RAM 10 (Kn,D & K)

// om nb>=RAM 10
// i annat fall nb+1

**2** Gör ett mini- M.a.o fixa

```
int getRank(int aCard) {
    printf("(%d) ",aCard);← Koll aCard ok!
    return 0;← Returnera ett fix legalt värde
}                tills koden kan räkna rätt!

int main(int argc, char **argv)
{
    printf("Monte Carlo 0.4\n");

    for (int card=0; card<DECK; card++) {
        printCard(card);
        printf("[%d]", getRank(card));
        printf("\n");
    }

    printf("Program completed, bye!\n");
    return 0;
}
```

```
Monte Carlo 0.4
[Club    A] (0) [0]
[Club    2] (1) [0]
[Club    3] (2) [0]
[Club    4] (3) [0]
[Club    5] (4) [0]
[Club    6] (5) [0]
[Club    7] (6) [0]
[Club    8] (7) [0]
[Club    9] (8) [0]
[Club    T] (9) [0]
[Club    J] (10) [0]
[Club    Q] (11) [0]
[Club    K] (12) [0]
[Diamond A] (13) [0]
[Diamond 2] (14) [0]
[Diamond 3] (15) [0]
[Diamond 4] (16) [0]
[Diamond 5] (17) [0]
[Diamond 6] (18) [0]
[Diamond 7] (19) [0]
[Diamond 8] (20) [0]
[Diamond 9] (21) [0]
[Diamond T] (22) [0]
[Diamond J] (23) [0]
[Diamond Q] (24) [0]
[Diamond K] (25) [0]
[Heart   A] (26) [0]
[Heart   2] (27) [0]
[Heart   3] (28) [0]
[Heart   4] (29) [0]
[Heart   5] (30) [0]
[Heart   6] (31) [0]
[Heart   7] (32) [0]
[Heart   8] (33) [0]
[Heart   9] (34) [0]
[Heart   T] (35) [0]
[Heart   J] (36) [0]
[Heart   Q] (37) [0]
[Heart   K] (38) [0]
[Spade   A] (39) [0]
[Spade   2] (40) [0]
[Spade   3] (41) [0]
[Spade   4] (42) [0]
[Spade   5] (43) [0]
[Spade   6] (44) [0]
[Spade   7] (45) [0]
[Spade   8] (46) [0]
[Spade   9] (47) [0]
[Spade   T] (48) [0]
[Spade   J] (49) [0]
[Spade   Q] (50) [0]
[Spade   K] (51) [0]
```

**Ok, det här ser bra ut… …dags för innehållet!**

# Nu behöver vi veta kortets värde?!

**0** Algoritmer & Datastrukturer

**2**

...det borde inte vara svårare än att returnera 10 för allt utom ess..9!

**1** **Skriv ner problemet så att ett barn förstår det!**

// RAM 10 (Kn,D & K)

// om nb>=RAM 10
// i annat fall nb+1

```
#define FACE  10

int getRank(int aCard) {
    if ((aCard%RANK)>=FACE)
        return FACE;
    else
        return (aCard%RANK)+1;
}

int main(int argc, char **argv)
{
    printf("Monte Carlo 0.5\n");

    for (int card=0; card<DECK; card++) {
        printCard(card);
        printf("[%d]", getRank(card));
        printf("\n");
    }

    printf("Program completed, bye!\n");
    return 0;
}
```

**3**

**4**

```
Monte Carlo 0.4
[Club    A] (0) [0]
[Club    2] (1) [0]
[Club    3] (2) [0]
[Club    4] (3) [0]
[Club    5] (4) [0]
[Club    6] (5) [0]
[Club    7] (6) [0]
[Club    8] (7) [0]
[Club    9] (8) [0]
[Club    T] (9) [0]
[Club    J] (10) [0]
[Club    Q] (11) [0]
[Club    K] (12) [0]
[Diamond A] (13) [0]
[Diamond 2] (14) [0]
[Diamond 3] (15) [0]
[Diamond 4] (16) [0]
[Diamond 5] (17) [0]
[Diamond 6] (18) [0]
[Diamond 7] (19) [0]
[Diamond 8] (20) [0]
[Diamond 9] (21) [0]
[Diamond T] (22) [0]
[Diamond J] (23) [0]
[Diamond Q] (24) [0]
[Diamond K] (25) [0]
[Heart   A] (26) [0]
[Heart   2] (27) [0]
[Heart   3] (28) [0]
[Heart   4] (29) [0]
[Heart   5] (30) [0]
[Heart   6] (31) [0]
[Heart   7] (32) [0]
[Heart   8] (33) [0]
[Heart   9] (34) [0]
[Heart   T] (35) [0]
[Heart   J] (36) [0]
[Heart   Q] (37) [0]
[Heart   K] (38) [0]
[Spade   A] (39) [0]
[Spade   2] (40) [0]
[Spade   3] (41) [0]
[Spade   4] (42) [0]
[Spade   5] (43) [0]
[Spade   6] (44) [0]
[Spade   7] (45) [0]
[Spade   8] (46) [0]
[Spade   9] (47) [0]
[Spade   T] (48) [0]
[Spade   J] (49) [0]
[Spade   Q] (50) [0]
[Spade   K] (51) [0]
```

```
Monte Carlo 0.5
[Club    A] [1]
[Club    2] [2]
[Club    3] [3]
[Club    4] [4]
[Club    5] [5]
[Club    6] [6]
[Club    7] [7]
[Club    8] [8]
[Club    9] [9]
[Club    T] [10]
[Club    J] [10]
[Club    Q] [10]
[Club    K] [10]
[Diamond A] [1]
[Diamond 2] [2]
[Diamond 3] [3]
[Diamond 4] [4]
[Diamond 5] [5]
[Diamond 6] [6]
[Diamond 7] [7]
[Diamond 8] [8]
[Diamond 9] [9]
[Diamond T] [10]
[Diamond J] [10]
[Diamond Q] [10]
[Diamond K] [10]
[Heart   A] [1]
[Heart   2] [2]
[Heart   3] [3]
[Heart   4] [4]
[Heart   5] [5]
[Heart   6] [6]
[Heart   7] [7]
[Heart   8] [8]
[Heart   9] [9]
[Heart   T] [10]
[Heart   J] [10]
[Heart   Q] [10]
[Heart   K] [10]
[Spade   A] [1]
[Spade   2] [2]
[Spade   3] [3]
[Spade   4] [4]
[Spade   5] [5]
[Spade   6] [6]
[Spade   7] [7]
[Spade   8] [8]
[Spade   9] [9]
[Spade   T] [10]
[Spade   J] [10]
[Spade   Q] [10]
[Spade   K] [10]
```

**Wow, en till bra funktion!**

1-dimentionella

# Matriser som parametrar

```
#include <stdio.h>
#define WEEKDAYS 7

int main(int argc, char **argv) {
    int box[WEEKDAYS]={2,1,1,1,0,0,1};
    printf("Matrix Tester (MT)!\n");

    for (int i=0; i<WEEKDAYS; i++)
        printf("Box[%d] : %d\n", i, box[i]);

    printf("MT Done, bye!\n");
    return 0;
}
```

Ett fåtal satser som hör ihop.
Kommer att upprepas.
(Bra för felsökning)

```
Matrix Tester (MT)!
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
Box[5] : 0
Box[6] : 1
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA  VÄRDE      NAMN          TYP
#F000:   2    box[0]           int
#F001:   1    box[1]           int
#F002:   1    box[2]           int
#F003:   1    box[3]           int
#F004:   0    box[4]           int
#F005:   0    box[5]           int
#F006:   1    box[6]           int
#F007:
#F008:
#F009:
```






Det är rätt troligt att vi vill skriva ut askens innehåll på flera ställen, vi skulle behöva en printBox()-funktion!

1-dimentionella

# Matriser som parametrar

```c
#include <stdio.h>
#define WEEKDAYS 7

void printBox(int aBox[], int size){
   for (int i=0; i<size; i++)
      printf("Box[%d] : %d\n", i, aBox[i]);
}

int main(int argc, char **argv) {
   int box[WEEKDAYS]={2,1,1,1,0,0,1};
   printf("Matrix Tester (MT)!\n");

   printBox(box, WEEKDAYS);

   printf("MT Done, bye!\n");
   return 0;
}
```

1 2

3

4

```
Matrix Tester (MT)!
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
Box[5] : 0
Box[6] : 1
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:   2    box[0] aBox[0]   int
#F001:   1    box[1] aBox[1]   int
#F002:   1    box[2] aBox[2]   int
#F003:   1    box[3] aBox[3]   int
#F004:   0    box[4] aBox[4]   int
#F005:   0    box[5] aBox[5]   int
#F006:   1    box[6] aBox[6]   int
#F007:   7    size             int
#F008:   0    i                int
#F009:
```

1: Parameter saknar storlek!
2: Storleken bifogas separat!
3: Argumentet saknar []
4: Den (till synes) nya var. aBox tilldelas **samma plats i minnet** som argumentet!!!
(Helt tvärt emot hur en int parameter skapades)



**!!! Skriver printBox() i variabel aBox kommer det efteråt att märkas i variabel box, används mycket ofta!!!**

1-dimentionella

# Matriser som parametrar

```c
#include <stdio.h>
#define WEEKDAYS 7
  #include <stdio.h>
in#define WEEKDAYS 7

  void printBox(int aBox[], int size){
     for (int i=0; i<size; i++)
        printf("Box[%d] : %d\n", i, aBox[i]);
  }

  int main(int argc, char **argv) {
     int box[WEEKDAYS]={2,1,1,1,0,0,1};
}    printf("Matrix Tester (MT)!\n");

     printBox(box, 5);

     printf("MT Done, bye!\n");
     return 0;
  }
```

```
Matrix Tester
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000:   2    box[0] aBox[0]   int
#F001:   1    box[1] aBox[1]   int
#F002:   1    box[2] aBox[2]   int
#F003:   1    box[3] aBox[3]   int
#F004:   0    box[4] aBox[4]   int
#F005:   0    box[5]           int
#F006:   1    box[6]           int
#F007:   5    size             int
#F008:   0    i                int
#F009:
```

Att det fungerar på detta sätt beror på att informationen som funk. får enbart är adressen till första elementet i matris. Variabeln box har värdet #F000! Funktionen har ingen aning om det är sant, eller hur matrisen faktiskt ser ut!



**!!! Det går utmärkt, och används ibland, att skicka med en ”falsk” (men ok) storlek !!!**

1-dimentionella

# Matriser som parametrar

```c
#include <stdio.h>
#define WEEKDAYS 7

void printBox(int aBox[], int size){
   for (int i=0; i<size; i++)
      printf("Box[%d] : %d\n", i, aBox[i]);
}

int main(int argc, char **argv) {
   int box[WEEKDAYS]={2,1,1,1,0,0,1};
   printf("Matrix Tester (MT)!\n");

   printBox(&box[3], 2);

   printf("MT Done, bye!\n");
   return 0;
}
```

```
Matrix Tester
Box[0] : 1
Box[1] : 0
MT Done, bye!
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000:   2    box[0]          int
#F001:   1    box[1]          int
#F002:   1    box[2]          int
#F003:   1    box[3] aBox[0]  int
#F004:   0    box[4] aBox[1]  int
#F005:   0    box[5]          int
#F006:   1    box[6]          int
#F007:   2    size            int
#F008:   0    i               int
#F009:
```

Det går utmärkt att skriva ut en ”bit” av en matris genom att ”påstå” att den börjar en bit in i den riktiga matrisen!

Det går att skapa namnlösa mat:

```
printBox((int []){1,0}, 2);
```

…otroligt bra för testning!



## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```c
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(b, S);
    do {
        printBoard(b, S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='O';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```



```c
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

void initBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            aB[row][col]='+';
    }
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000:   ?   b[0][0]          char
#F001:   ?   b[0][1]          char
#F002:   ?   b[0][2]          char
#F003:   ?   b[1][0]          char
#F004:   ?   b[1][1]          char
#F005:   ?   b[1][2]          char
#F006:   ?   b[2][0]          char
#F007:   ?   b[2][1]          char
#F008:   ?   b[2][2]          char
#F009: '0'   r                char
#F00A: '0'   c                char
#F00B:   0   turn             int


Allt lugnt…
 …förväntade variabler skapade!
```



Vi tar en ny titt på Tic tac toe programmet, och skapar oss rutinerna printBoard() och initBoard()!

## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```c
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(b, S);
    do {
        printBoard(b, S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='0';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```



```c
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

1 2

void initBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            aB[row][col]='+';
    }
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000:   ?   b[0][0]ab[0][0] char
#F001:   ?   b[0][1]ab[0][1] char
#F002:   ?   b[0][2]ab[0][2] char
#F003:   ?   b[1][0]ab[1][0] char
#F004:   ?   b[1][1]ab[1][1] char
#F005:   ?   b[1][2]ab[1][2] char
#F006:   ?   b[2][0]ab[2][0] char
#F007:   ?   b[2][1]ab[2][1] char
#F008:   ?   b[2][2]ab[2][2] char
#F009: '0'   r               char
#F00A: '0'   c               char
#F00B:   0   turn            int
#F00C:   3   rows            int

        VIKTIGT!

1) 1:a dimensionen enbart []
2) 2:a dimensionen storlek [3]
```



Observera hur matrisparam. hamnar på samma ställe som argumentet, medan int param. får en ny minnescell!



## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```c
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(b, S);
    do {
        printBoard(b, S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='0';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```

```c
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

void initBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            aB[row][col]='+';
    }
}
```

!

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000: '+' b[0][0]ab[0][0] char
#F001:  ?  b[0][1]ab[0][1] char
#F002:  ?  b[0][2]ab[0][2] char
#F003:  ?  b[1][0]ab[1][0] char
#F004:  ?  b[1][1]ab[1][1] char
#F005:  ?  b[1][2]ab[1][2] char
#F006:  ?  b[2][0]ab[2][0] char
#F007:  ?  b[2][1]ab[2][1] char
#F008:  ?  b[2][2]ab[2][2] char
#F009: '0' r               char
#F00A: '0' c               char
#F00B:  0  turn            int
#F00C:  3  rows            int
#F00D:  0  row             int
#F00E:  0  col             int
```

**MAGISKT!**



**Observera hur koden skriver ett ’+’ till aB[0][0] men i själva verket skriver den till b[0][0]!**



## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(b, S);
    do {
        printBoard(b, S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
         else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='O';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```



```
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

void initBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            aB[row][col]='+';
    }
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE      NAMN          TYP
#F000: '+' b[0][0]            char
#F001: '+' b[0][1]            char
#F002: '+' b[0][2]            char
#F003: '+' b[1][0]            char
#F004: '+' b[1][1]            char
#F005: '+' b[1][2]            char
#F006: '+' b[2][0]            char
#F007: '+' b[2][1]            char
#F008: '+' b[2][2]            char
#F009: '0' r                  char
#F00A: '0' c                  char
#F00B:  0  turn               int
#F00C:
#F00D:
#F00E:

initBoard() är klar och alla
variabler relaterade till
funktionen är raderade, men…
```



**…mainfunktionens matrisvariabel b[][] är nu korrekt initierad!**



## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(&b[0][0], S*S);
    do {
        printBoard(&b[0][0], S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='0';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```

```
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

void initBoard(char aB[], int rows){
    for (int row=0; row<rows; row++) {
        aB[row]='+';
    }
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000: '+' b[0][0]ab[0]     char
#F001:  ?  b[0][1]ab[1]     char
#F002:  ?  b[0][2]ab[2]     char
#F003:  ?  b[1][0]ab[3]     char
#F004:  ?  b[1][1]ab[4]     char
#F005:  ?  b[1][2]ab[5]     char
#F006:  ?  b[2][0]ab[6]     char
#F007:  ?  b[2][1]ab[7]     char
#F008:  ?  b[2][2]ab[8]     char
#F009: '0' r                char
#F00A: '0' c                char
#F00B:  0  turn             int
#F00C:  9  rows             int
#F00D:  0  row              int
```

**MAGISKT!**

…vi vet hur elementen är utlagda och ”lurar” initBoard() att tro att det är en 1-dim array med 9 element!



## 2-dimentionella

# Matriser som parametrar

### Tic tac toe

```
int main(int argc, char **argv){
    char b[S][S];
    char r='0',c='0'; int turn=0;

    printf("Tic Tac Toe!\n");
    printf("Enter rowcol\n");
    printf("or 00 to end\n");
    initBoard(&b[0][0], S*S);
    do {
        printBoard(&b[0][0], S);

        printf("Where?");
        scanf(" %c%c",&r,&c);

        if (r<'1' || c<'1')
            turn=0;
        else {
            if (++turn%2)
                b[r-'1'][c-'1']='X';
            else
                b[r-'1'][c-'1']='0';
        }
    } while (turn);
    printf("Done, bye!\n");
    return 0;
}
```

```
#include <stdio.h>
#define S 3

void printBoard(char aB[][S], int rows){
    for (int row=0; row<rows; row++) {
        for (int col=0; col<S; col++)
            printf("%c", aB[row][col]);
        printf("\n");
    }
}

void initBoard(char aB[], int rows){
    for (int row=0; row<rows; row++) {
        aB[row]='+';
    }
}
```

```
 PROGRAM
#0000: CODE
   *
 DATA VÄRDE     NAMN          TYP
#F000: '+' b[0][0]ab[0]     char
#F001:  ?  b[0][1]ab[1]     char
#F002:  ?  b[0][2]ab[2]     char
#F003:  ?  b[1][0]ab[3]     char
#F004:  ?  b[1][1]ab[4]     char
#F005:  ?  b[1][2]ab[5]     char
#F006:  ?  b[2][0]ab[6]     char
#F007:  ?  b[2][1]ab[7]     char
#F008:  ?  b[2][2]ab[8]     char
#F009: '0' r                char
#F00A: '0' c                char
#F00B:  0  turn             int
#F00C:  9  rows             int
#F00D:  0  row              int

b[0] (b[1] eller b[2]) kan anv.
som en 1-dim matris med 3 ele-
ment, mycket mycket vanligt!
```



**Att förstå hur int/float/char/[] fungerar som parametrar är avgörande för att klara kursen!**



# Monte Carlo 0.9

```
Monte Carlo 0.9
[Heart   2]  2 h(it) or s(top) ? h
[Spade   J] 12 h(it) or s(top) ? h
[Diamond T] 22 BUSTED!
Program completed, bye!
ACMacBookPro:Debug and144$ cd /User
ris/Debug && ./paris
Monte Carlo 0.9
[Diamond T] 10 h(it) or s(top) ? h
[Diamond J] 20 h(it) or s(top) ? s
Program completed, bye!
ACMacBookPro:Debug and144$ cd /User
ris/Debug && ./paris
Monte Carlo 0.9
[Heart   3]  3 h(it) or s(top) ? h
[Spade   5]  8 h(it) or s(top) ? h
[Club    4] 12 h(it) or s(top) ? h
[Spade   8] 20 h(it) or s(top) ? s
Program completed, bye!
```

**Önskemål om funktion skapad i dialog med slutanvändare**

**initRandomGenerator()**
**Återanvändbara slumpfunktioner!**
**rndNumber(52)**

**printCard(theCard)**
**Återanvändbara kortfunktioner!**
**getRank(theCard)**

**Hum… Inga speciella algoritmer eller data-strukturer, 4 bra rutiner finns!**

Algoritmer & Datastrukturer **0**

// Välkommen!
// Initiera slumpgeneratorn (blanda kortleken)…

**1** **Skriv ner proble-met så att ett barn förstår det!**

**Gör ett minimalt main() av dina anteckningar!** **2**

**3** **a..n** **Konstanter Variabler Uttryck & resultat!**

**Läs in riktiga värden!** **4**

// klart!



# Monte Carlo 0.9

initRandomGenerator()

Återanvändbara slumpfunktioner!

rndNumber(52)

printCard(theCard)

Återanvändbara kortfunktioner!

getRank(theCard)

```
Monte Carlo 0.9
 [Heart   2]  2 h(it) or s(top) ? h
 [Spade   J] 12 h(it) or s(top) ? h
 [Diamond T] 22 BUSTED!
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Diamond T] 10 h(it) or s(top) ? h
 [Diamond J] 20 h(it) or s(top) ? s
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Heart   3]  3 h(it) or s(top) ? h
 [Spade   5]  8 h(it) or s(top) ? h
 [Club    4] 12 h(it) or s(top) ? h
 [Spade   8] 20 h(it) or s(top) ? s
 Program completed, bye!
```

Önskemål om funktion skapad i dialog med slutanvändare

```
// heltal card
// heltal player = 0

// Välkommen!
// Initiera slumpgeneratorn…


//    Dra ett kort (card)
//    Addera värdet till player
//    Visa kortet…
//    ...och värdet så här långt!




// klart!
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**



# Monte Carlo 0.9

initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

```
Monte Carlo 0.9
 [Heart   2]  2 h(it) or s(top) ? h
 [Spade   J] 12 h(it) or s(top) ? h
 [Diamond T] 22 BUSTED!
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Diamond T] 10 h(it) or s(top) ? h
 [Diamond J] 20 h(it) or s(top) ? s
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Heart   3]  3 h(it) or s(top) ? h
 [Spade   5]  8 h(it) or s(top) ? h
 [Club    4] 12 h(it) or s(top) ? h
 [Spade   8] 20 h(it) or s(top) ? s
 Program completed, bye!
```

Önskemål om funktion skapad i dialog med slutanvändare

```
// teckenvariabel cmd
// heltal card
// heltal player = 0

// Välkommen!
// Initiera slumpgeneratorn…


//   Dra ett kort (card)
//   Addera värdet till player
//   Visa kortet…
//   ...och värdet så här långt!
//   om (inte tjock)
//     ledtext hit eller stop
//     läs in val (cmd)
//   i annat fall
//     skriv ut tjock
//     cmd='s'

// klart!
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**



# Monte Carlo 0.9

```
Monte Carlo 0.9
 [Heart   2]  2 h(it) or s(top) ? h
 [Spade   J] 12 h(it) or s(top) ? h
 [Diamond T] 22 BUSTED!
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Diamond T] 10 h(it) or s(top) ? h
 [Diamond J] 20 h(it) or s(top) ? s
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Heart   3]  3 h(it) or s(top) ? h
 [Spade   5]  8 h(it) or s(top) ? h
 [Club    4] 12 h(it) or s(top) ? h
 [Spade   8] 20 h(it) or s(top) ? s
 Program completed, bye!
```

**Önskemål om funktion skapad i dialog med slutanvändare**

```
// teckenvariabel cmd
// heltal card
// heltal player = 0

// Välkommen!
// Initiera slumpgeneratorn…

// Repetera...
//    Dra ett kort (card)
//    Addera värdet till player
//    Visa kortet…
//    ...och värdet så här långt!
//    om (inte tjock)
//       ledtext hit eller stop
//       läs in val (cmd)
//    i annat fall
//       skriv ut tjock
//       cmd='s'
//  ...så länge 'h'
// klart!
```

**initRandomGenerator()**
**Återanvändbara slumpfunktioner!**
**rndNumber(52)**

**printCard(theCard)**
**Återanvändbara kortfunktioner!**
**getRank(theCard)**

Algoritmer & Datastrukturer **0**

**1** **Skriv ner proble-met så att ett barn förstår det!**

**Gör ett minimalt main() av dina anteckningar!** **2**

**3** **a..n** **Konstanter Variabler Uttryck & resultat!**

**Läs in riktiga värden!** **4**

# Monte Carlo 0.9

initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

```
Monte Carlo 0.9
 [Heart   2]  2 h(it) or s(top) ? h
 [Spade   J] 12 h(it) or s(top) ? h
 [Diamond T] 22 BUSTED!
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Diamond T] 10 h(it) or s(top) ? h
 [Diamond J] 20 h(it) or s(top) ? s
 Program completed, bye!
[ACMacBookPro:Debug and144$ cd /User
 ris/Debug && ./paris
 Monte Carlo 0.9
 [Heart   3]  3 h(it) or s(top) ? h
 [Spade   5]  8 h(it) or s(top) ? h
 [Club    4] 12 h(it) or s(top) ? h
 [Spade   8] 20 h(it) or s(top) ? s
 Program completed, bye!
```

Önskemål om funktion skapad i dialog med slutanvändare

```
int main(int argc, char **argv) {
    //char cmd
    //int card
    //int player

    printf("Monte Carlo 0.91\n");
    //initRandomGenerator();

    //Repetera...
    //  Dra ett kort
    //  Addera värdet till player
    //  Visa kortet
    //  Visa accumulerade värdet
    //  om inte tjock
    //    ledtext hit eller stop
    //    läs in val
    //  i annat fall
    //     skriv ut tjock
    //     val är stop
    //...så länge hit

    printf("Program completed, bye!\n");
    return 0;
}
```

```
Monte Carlo 0.91
Program completed, bye!
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**



Perfekt, ett fungerande program! Vad är ett lämpligt nästa steg?

# Monte Carlo 0.9

initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define RANK    13
#define FACE    10

void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}

int rndNumber(int nbs){
    return rand()%nbs;
}

void printCard(int aCard){
    switch (aCard/RANK) {
        case  0:printf("[Club   "); break;
        case  1:printf("[Diamon "); break;
        case  2:printf("[Heart  "); break;
        case  3:printf("[Spade  "); break;
    }
    switch (aCard%RANK) {
        case  0:printf("A] "); break;
        case  9:printf("T] "); break;
        case 10:printf("J] "); break;
        case 11:printf("Q] "); break;
        case 12:printf("K] "); break;
        default:printf("%d] ",(aCard%RANK)+1); break;
    }
}

int  getRank(int aCard){
    if ((aCard%RANK)>=FACE)
        return FACE;
    else
        return (aCard%RANK)+1;
}
```

```
int main(int argc, char **argv) {
    //char cmd
    //int card
    //int player

    printf("Monte Carlo 0.92\n");
    initRandomGenerator();

    //Repetera...
    //  Dra ett kort
    //  Addera värdet till player
    //  Visa kortet
    //  Visa accumulerade värdet
    //  om inte tjock
    //    ledtext hit eller stop
    //    läs in val
    //  i annat fall
    //     skriv ut tjock
    //     val är stop
    //...så länge hit

    printf("Program completed, bye!\n");
    return 0;
}
```

```
Monte Carlo 0.92
Program completed, bye!
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Lyft in de återanvändbara funktionerna och testa igen! Vad är ett lämpligt nästa steg?

# Monte Carlo 0.9

initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define RANK    13
#define FACE    10

void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}

int rndNumber(int nbs){
    return rand()%nbs;
}

void printCard(int aCard){
    switch (aCard/RANK) {
        case  0:printf("[Club   "); break;
        case  1:printf("[Diamon "); break;
        case  2:printf("[Heart  "); break;
        case  3:printf("[Spade  "); break;
    }
    switch (aCard%RANK) {
        case  0:printf("A] "); break;
        case  9:printf("T] "); break;
        case 10:printf("J] "); break;
        case 11:printf("Q] "); break;
        case 12:printf("K] "); break;
        default:printf("%d] ",(aCard%RANK)+1); break;
    }
}

int  getRank(int aCard){
    if ((aCard%RANK)>=FACE)
        return FACE;
    else
        return (aCard%RANK)+1;
}
```

```c
int main(int argc, char **argv) {
    //char cmd
    int card;
    int player=0;

    printf("Monte Carlo 0.92\n");
    initRandomGenerator();

    //Repetera...
        card = rndNumber(DECK);
        player=player+getRank(card);
        printCard(card);
        printf("%2d ", player);
```

```
Monte Carlo 0.93
[Diamon 2]  2 Program completed, bye!
Monte Carlo 0.93
[Club   2]  2 Program completed, bye!
Monte Carlo 0.93
[Heart  T] 10 Program completed, bye!
```

```c
    printf("Program completed, bye!\n");
    return 0;
}
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**

Fungerar bra! Vad är ett lämpligt nästa steg?

# Monte Carlo 0.9

initRandomGenerator()
Återanvändbara slumpfunktioner!
rndNumber(52)

printCard(theCard)
Återanvändbara kortfunktioner!
getRank(theCard)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define RANK    13
#define FACE    10

void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}
int rndNumber(int nbs){
    return rand()%nbs;
}

void printCard(int aCard){
    switch (aCard/RANK) {
        case  0:printf("[Club   "); break;
        case  1:printf("[Diamon "); break;
        case  2:printf("[Heart  "); break;
        case  3:printf("[Spade  "); break;
    }
    switch (aCard%RANK) {
        case  0:printf("A] "); break;
        case  9:printf("T] "); break;
        case 10:printf("J] "); break;
        case 11:printf("Q] "); break;
        case 12:printf("K] "); break;
        default:printf("%d] ",(aCard%RANK)+1); break;
    }
}

int  getRank(int aCard){
    if ((aCard%RANK)>=FACE)
        return FACE;
    else
        return (aCard%RANK)+1;
}
```

```
int main(int argc, char **argv) {
    char cmd='s';
    int card;
    int player=0;

    printf("Monte Carlo 0.94\n");
    initRandomGenerator();

    //Repetera...
        card = rndNumber(DECK);
        player=player+getRank(card);
        printCard(card);
        printf("%2d ", player);
        if (player<BUSTED) {
            printf("h(it) or s(top) ? ");
    //      Läs in val
        } else {
            printf("BUSTED!\n");
            cmd='s';
        }
    //...så länge hit
    Monte Carlo 0.94
    [Heart  T] 10 h(it) or s(top) ? Program completed, bye!
    return 0;
}
```

Algoritmer & Datastrukturer **0**

**1** Skriv ner problemet så att ett barn förstår det!

Gör ett minimalt main() av dina anteckningar! **2**

**3** a..n Konstanter Variabler Uttryck & resultat!

Läs in riktiga värden! **4**



Fungerar bra! Vad är ett lämpligt nästa steg?

# Monte Carlo 0.9

initRandomGenerator()

Återanvändbara slumpfunktioner!

rndNumber(52)

printCard(theCard)

Återanvändbara kortfunktioner!

getRank(theCard)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define RANK    13
#define FACE    10

void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}
int rndNumber(int nbs){
    return rand()%nbs;
}

void printCard(int aCard){
    switch (aCard/RANK) {
        case  0:printf("[Club   "); break;
        case  1:printf("[Diamon "); break;
        case  2:printf("[Heart  "); break;
        case  3:printf("[Spade  "); break;
    }
    switch (aCard%RANK) {
        case  0:printf("A] "); break;
        case  9:printf("T] "); break;
        case 10:printf("J] "); break;
        case 11:printf("Q] "); break;
        case 12:printf("K] "); break;
        default:printf("%d] ",(aCard%RANK)+1); break;
    }
}

int  getRank(int aCard){
    if ((aCard%RANK)>=FACE)
        return FACE;
    else
        return (aCard%RANK)+1;
}
```

```c
int main(int argc, char **argv) {
    char cmd='s';
    int card;
    int player=0;

    printf("Monte Carlo 0.95\n");
    initRandomGenerator();

    do {
        card = rndNumber(DECK);
        player=player+getRank(card);
        printCard(card);
        printf("%2d ", player);
        if (player<BUSTED) {
            printf("h(it) or s(top) ? ");
            scanf(" %c", &cmd);
        } else {
            printf("BUSTED!\n");
            cmd='s';
        }
    } while (cmd=='h');

    printf("Program completed, bye!\n");
    return 0;
}
```

Algoritmer & Datastrukturer **0**

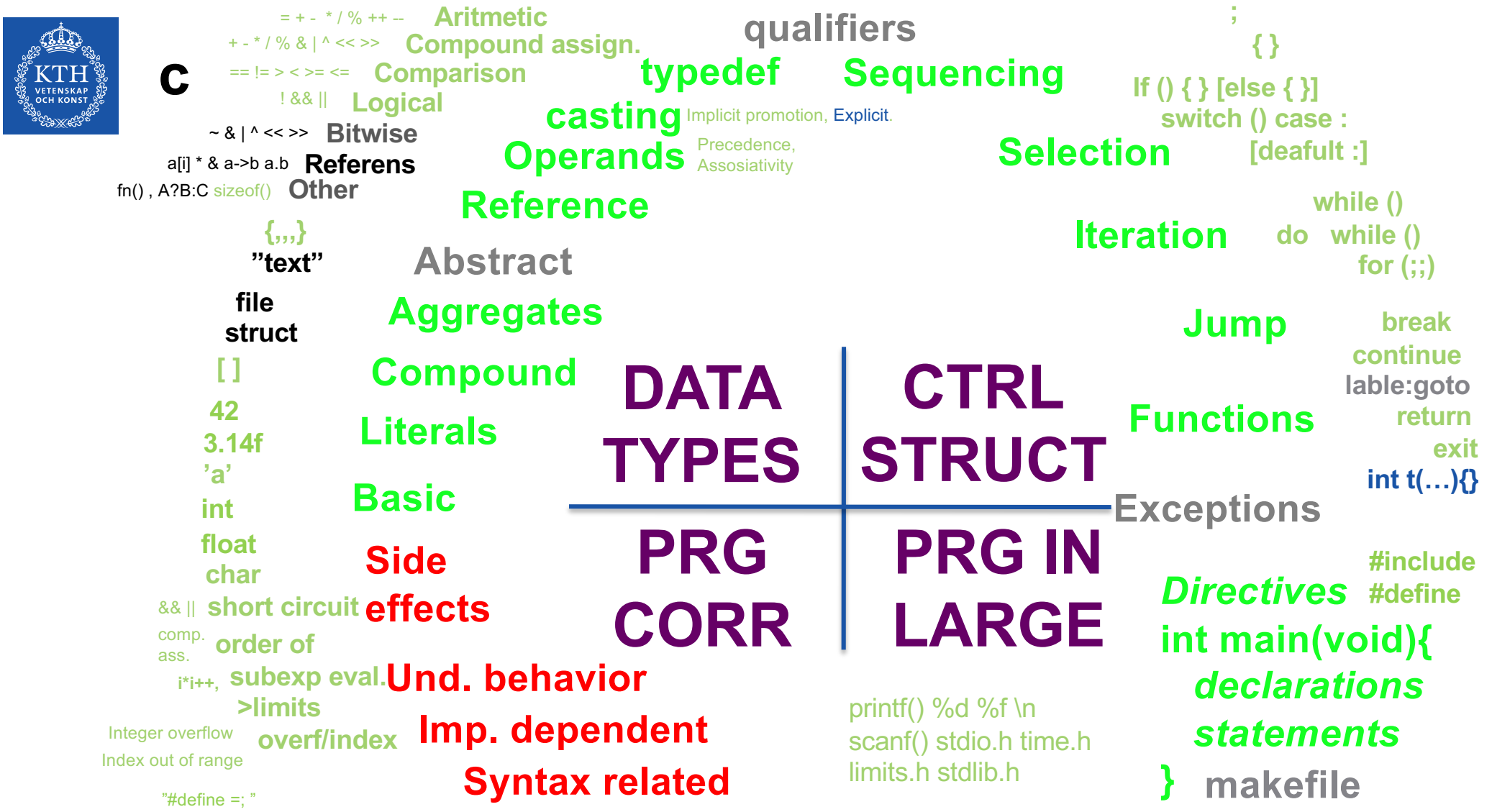
```
Monte Carlo 0.95
[Heart  7]  7 h(it) or s(top) ? h
[Club   7] 14 h(it) or s(top) ? h
[Club   A] 15 h(it) or s(top) ? h
[Diamon Q] 25 BUSTED!
Program completed, bye!
```

**Gör ett minimalt main() av dina anteckningar!** **2**

```
Monte Carlo 0.95
[Diamon J] 10 h(it) or s(top) ? h
[Club   A] 11 h(it) or s(top) ? h
[Club   T] 21 h(it) or s(top) ? s
Program completed, bye!
Hit any key to continue...
```

**Läs in riktiga värden!** **4**

Wow!


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
Operands
casting
Implicit promotion, Explicit.
Precedence,
Assosiativity
typedef
qualifiers
Sequencing
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
} makefile
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
&& || short circuit
comp. ass.
order of
subexp eval.
i*i++,
Side effects
Und. behavior
Imp. dependent
Syntax related
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
int/int
/*comment*/ or //



# Övn #7: Monte Carlo 1.0

```
Monte Carlo 1.0
[Spade  K] 10 h(it) or s(top) ? h
[Heart  Q] 20 h(it) or s(top) ? s
[Heart  J] 10 h(it) or s(top) ? h
[Spade  9] 19 h(it) or s(top) ? h
[Heart  9] 28 BUSTED!
*** WINN :-)
Program completed, bye!
```

Player

Bank

```
Monte Carlo 1.0
[Spade  Q] 10 h(it) or s(top) ? h
[Diamon 3] 13 h(it) or s(top) ? h
[Spade  4] 17 h(it) or s(top) ? s
[Heart  7]  7 h(it) or s(top) ? h
[Diamon 6] 13 h(it) or s(top) ? h
[Heart  2] 15 h(it) or s(top) ? h
[Diamon 5] 20 h(it) or s(top) ? s
*** LOSE :-(
Program completed, bye!
```

**1** Skriv ner problemet så att ett barn förstår det!

**2** Gör ett minimalt main() av dina anteckningar!

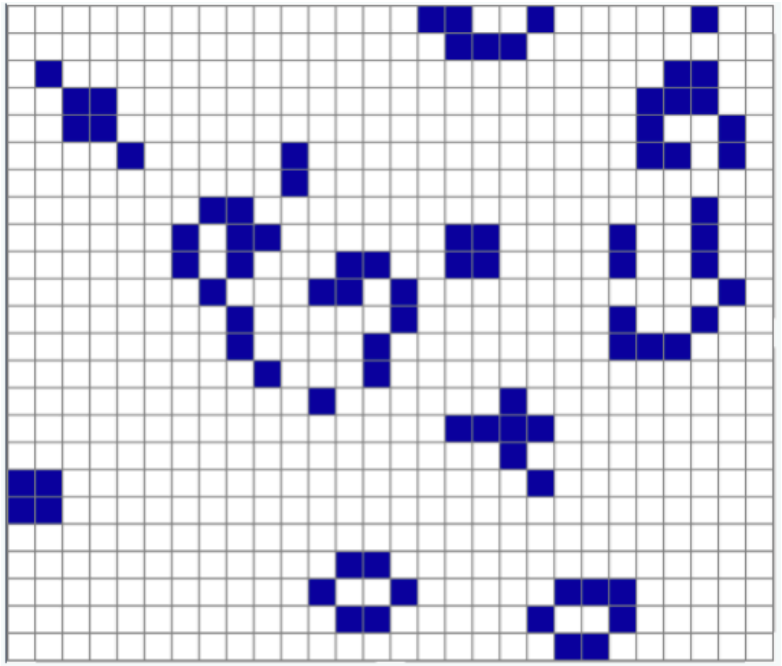
**3** Konstanter
Variabler med
Värde Uttryck
**a..n** & resultat!

**4** Läs in riktiga värden!



Banken får inte stanna innan 17, får inte ta ett till kort om mer än 16 och samma som spelaren (DRAW).

# Övning #7: Life (Utmaning!)



The rules for *Game of Life* are simple. They determine when a live cell remains alive from one generation to the next, when a living cell dies, and when an empty grid point can birth a new cell. The number of surrounding grid points, or neighbors (which will be 8 or fewer), that hold a live cell is summed and that value is used to determine what is contained in the surrounded cell in the next generation according to the following:

- A cell that is alive remains alive if there are 2 or 3 live neighbors,
- A live cell dies if the number of live neighbors is less than 2 (loneliness) or greater than 3 (starved for food), and
- An empty (dead) cell will birth a new alive cell if there are exactly three live neighbors.

**0** Algoritmer & Datastrukturer

Det behövs en 2-dim kvadratisk matris av storleken SIZE 15

**1** **Skriv ner problemet så att ett barn förstår det!**

Tänk att du har printHabitat()
Tänk att du har initHabitat()
Tänk att du har nextHabitat()

**2** **Gör ett minimalt main() av dina anteckningar!**

**3** a..n **Konstanter Variabler Uttryck & resultat!**

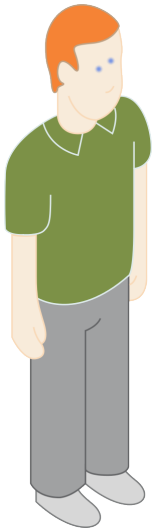
Skapa matrisen och printHabitat()
Utveckla initHabitat()*
Utveckla nextHabitat()

Tips →

```
initialize grids;
while (more generations) do {
   for i = 1,maxrow
      for j = 1,maxcol {
            count neighbors of cell(i,j) from current generation;
            set next generation status of cell(i,j);
      }
   make next generation the current generation;
}
```

*) Sätt initialt en konstant till 5% chans att positionen lever, fråga efter %-satsen i steg 4, kör t.ex. 20 generationer!

# Historik



| | | |
|---|---|---|
| • 150719 0.1 | AC | Skapad. |
| • 150720 0.2 | AC | rep |
| • 150721 0.5 | AC | funk |
| • 150722 0.9 | AC | clean up |
| • 150723 1.0 | AC | övning |
| • 160908 1.1 | AC | Mindre förbättringar |
| • 190801 2.0 | AC | Omarbetad från grunden, samt anpassad till KTH:s nya profil. |